

Instance segmentation of lunar surface features with a custom Mask R-CNN

Computational Physics 2026 – Moon feature recognition

Amirmohammad Saiedi Saber¹

Department of Physics and Astronomy “G. Galilei”, University of Padua, Via Marzolo 8, 35131 Padova, Italy
e-mail: amirmohammad.saiedisaber@studenti.unipd.it

Laboratory of Computational Physics (LCP) module B – Physics of Data, 2026

ABSTRACT

Context. Automated mapping of geological features on the lunar surface is becoming essential as the volume of high-resolution orbital imagery grows faster than it can be inspected by hand. Small features such as collapse pits and possible lava-tube skylights are of particular interest for future exploration of the Moon, yet they are sparse and easy to miss.

Aims. This part of the project asks whether a single instance-segmentation network can both *detect* and *outline* several classes of lunar features — impact craters, pit/skylight candidates, wrinkle ridges and lobate scarps — directly from pre-processed image tiles of the Marius Hills region.

Methods. I convert the multi-channel semantic masks supplied with the data into per-object instance targets with connected-component labelling, and train a Mask R-CNN (ResNet-50 + FPN backbone, COCO-pretrained) in which I replace the standard detection and mask heads with deeper custom heads, retune the anchor scales to the size of lunar features, and keep the native 256×256 tile resolution. The whole pipeline was trained on CPU with a crash-safe, resumable loop.

Results. On a spatially held-out validation band the model reaches a mean mask IoU of 0.64 and Dice of 0.77 on the objects it does match, showing that the segmentation branch learns sensible feature outlines. Detection, however, is the bottleneck: mean average precision at IoU 0.5 is only $\sim 1\%$, with high false-negative counts driven by severe class imbalance and the noisy semantic-to-instance conversion.

Conclusions. The experiment shows that mask quality and detection quality decouple sharply on this dataset: the model can draw a feature once it is found, but finding the rare features reliably is the hard part. I close with the concrete data and compute challenges behind this gap and the changes I would make next.

Key words. Moon – instance segmentation – Mask R-CNN – planetary surfaces – deep learning – lava-tube skylights

1. Introduction

The Moon is being re-surveyed at a level of detail that no team of human analysts can keep up with. The Lunar Reconnaissance Orbiter Camera (LROC) carries a Wide-Angle Camera (WAC) for global, multi-band context mapping and two Narrow-Angle Cameras (NAC) that resolve the surface down to roughly half a metre per pixel ([USGS Astrogeology Science Center & Arizona State University 2024](#)). It is in NAC imagery that the now-famous Marius Hills pit was identified — a steep-walled hole tens of metres across that is widely interpreted as a skylight into a collapsed lava tube ([NASA/JPL-Caltech & University of Arizona 2010](#)). Features of this kind, together with the permanently shadowed terrain near the lunar south pole now being imaged by ShadowCam ([NASA/GSFC & Arizona State University 2011](#); [NASA 2023](#)), are exactly the targets that motivate automated detection: they are scientifically valuable, operationally relevant for future landings, and far too rare to find by eye at survey scale.

My part of the project tackles this as an *instance segmentation* problem rather than plain classification or detection: I want, for every tile, a list of individual feature objects, each with its own class label, bounding box and pixel-level mask. Mask R-CNN ([He et al. 2017](#)) is the natural tool, because it predicts a segmentation mask in parallel with the detection of each object.

The rest of this section reports how I adapted it to lunar tiles, what the network actually achieved, and — most importantly for a first attempt — where it struggled and why.

2. Data and instance-target preparation

The data I worked with are pre-processed tiles of the Marius Hills region supplied for the course ([Laboratory of Computational Physics \(LCP\) module B, Physics of Data, UniPD 2026](#)), stored as NPZ files with an accompanying metadata index (`index.csv`). Each tile holds an image array of shape $3 \times 256 \times 256$ and a mask array of shape $7 \times 256 \times 256$: seven *semantic* channels, one per annotated feature type, in the order impact crater, pit/skylight, wrinkle ridge, lobate scarp, irregular mare patch, Apollo site and candidate rille. The imagery is effectively single-band lunar grayscale, which I replicate across three channels so that the RGB-pretrained backbone can ingest it.

Channel audit. A semantic channel is not the same thing as a set of objects. Before training I scanned every tile and measured, per channel, in how many tiles it is labelled and what fraction of pixels it covers. This audit drives two decisions. First, I keep only channels that are labelled in at least 200 tiles, so the network

trains on features that are frequent enough to learn; rarer channels are dropped. Second, I flag channels whose mean coverage exceeds 20% of the tile as “coverage-like” rather than object-like — these behave more like background/terrain masks than countable objects and cannot be split into instances cleanly. In practice I trained on four classes: *impact crater*, *pit/skylight*, *wrinkle ridge* and *lobate scarp*.

From semantic masks to instances. Mask R-CNN needs one target per object, so I convert each selected semantic channel into instances with connected-component labelling (8-connectivity). Every connected blob becomes one object with a binary mask, a tight bounding box, a class label and an area. I discard components smaller than 30 px (speckle noise) or larger than 6000 px (merged blobs that cannot be separated into real objects), drop slivers narrower than 3 px on a side, and cap the number of instances per tile at 100. This step is deliberately conservative: noisy targets here propagate directly into poor detections later, a point I return to in Sect. 6.

Spatial split and augmentation. Neighbouring tiles share illumination and morphology, so a random train/validation split would leak almost-identical context across the two sets. I instead hold out a spatial band — the right-most tile columns, read from the `rXXXXX_cXXXXX` coordinates encoded in each filename — and use a random split only as a fallback. Training tiles are augmented with geometric transforms (flips/rotations) applied to both image and masks, plus brightness/contrast jitter applied to the image only.

3. Network architecture

The backbone is a ResNet-50 with a Feature Pyramid Network (He et al. 2016; Lin et al. 2017), initialised from COCO-pretrained Mask R-CNN weights, with the lowest layers frozen and the top three trainable so that transfer learning adapts the high-level features without destroying the pretrained low-level filters. On top of this backbone I made three changes that form my main architectural contribution.

Lunar-scale anchors. Lunar features in these tiles are small and span a wide size range, unlike the object scales COCO anchors are tuned for. I set one anchor size per FPN level, (8, 16, 32, 64, 128) pixels, each with aspect ratios (0.5, 1.0, 2.0). Using a single size per level keeps three anchors per location, which matches the shape of the pretrained Region Proposal Network head so its weights can be reused rather than reinitialised.

Deeper detection head. I replace the default two-layer box head with a four-layer fully-connected head (1024 units per layer, dropout 0.10) operating on the 7×7 pooled RoI features, followed by the standard classification and box-regression predictors for the (background + four) classes.

Deeper residual mask head. For segmentation I use a residual convolutional mask branch: a stem convolution followed by four residual blocks, each two 3×3 convolutions with Group-Norm and a skip connection (256 channels, dropout 0.05), which makes it safe to stack depth without destabilising training. A mask predictor then refines the features, upsamples $14 \rightarrow 28$

with a transposed convolution, and outputs per-class mask logits. All new layers use Kaiming initialisation.

Finally, I keep the native 256 px tile resolution instead of letting torchvision rescale to 800 px, and at inference time I use a low score threshold (0.05, to keep enough detections for the precision/recall curves), NMS at 0.5, and up to 200 detections per tile.

4. Training setup

Optimisation uses AdamW (Loshchilov & Hutter 2019) with a base learning rate of 2×10^{-4} and *discriminative* learning rates — the pretrained backbone and RPN train at one tenth of the head learning rate, so the new heads adapt quickly while the transferred features move gently. I use weight decay 10^{-4} , a one-epoch linear warm-up followed by cosine decay, and gradient clipping at 5.0. Training ran for 20 epochs with batch size 8.

Because the only hardware available to me was a CPU, a single epoch took on the order of half an hour and the full run roughly half a day. I therefore built the training loop to be *crash-safe and resumable*: every epoch writes the full training history and a complete `last_model.pth` checkpoint (model, optimiser, scheduler and scaler state), while `best_model.pth` always holds the best-mAP weights seen so far. If the machine is interrupted, re-running the cell continues from the last completed epoch. Mixed precision is requested but silently becomes a no-op on CPU, and per-epoch validation is capped to keep monitoring fast; the final evaluation reported below uses the entire validation set.

5. Results

I evaluate the best checkpoint on the held-out spatial band with object-level metrics: mean average precision at IoU 0.5 (per class, then averaged), precision/recall/F1 on confident detections, mean mask IoU and Dice over matched objects, and a count error. Table 1 collects the headline numbers and Table 2 breaks the average precision down by class.

Table 1. Final validation metrics (best checkpoint, full validation set).

Metric	Value
mAP@0.5	0.010
Precision / Recall / F1	0.104 / 0.056 / 0.073
Mean mask IoU (matched)	0.636
Mean mask Dice (matched)	0.773
Semantic-union Dice / IoU	0.113 / 0.070
Count MAE (objects per tile)	6.30
True / False pos. / False neg.	461 / 3973 / 7790

Table 2. Average precision at IoU 0.5 per class.

Class	AP@0.5
Impact crater	0.0233
Wrinkle ridge	0.0172
Pit / skylight	0.0003
Lobate scarp	0.0000

The two halves of the network tell very different stories. The *segmentation* branch works: whenever a detection is correctly matched to a ground-truth object, the predicted mask overlaps it

well (IoU 0.64, Dice 0.77). The *detection* branch is where everything breaks down — mAP@0.5 sits near 1%, recall is low, and false negatives (7790) vastly outnumber true positives (461). The per-class breakdown is even more telling: the comparatively common impact craters and wrinkle ridges carry essentially all of the average precision, the scientifically interesting pit/skylight class is barely detected, and lobate scarps reach exactly zero — there were too few of them for the model to learn anything.

Figure 1 shows why training did not rescue this. Training loss falls steadily across all 20 epochs, but validation loss bottoms out around epoch 4 and then climbs — a clear, early onset of overfitting despite dropout and weight decay. Validation mAP peaks near epoch 8 at about 0.014 and then drifts sideways, while the mask IoU jumps to ~ 0.6 almost immediately and stays flat. In other words, the model learned to draw masks within the first epoch or two and spent the remaining epochs slowly memorising the training set without improving detection.

The qualitative panels in Fig. 2 are consistent with the numbers: on tiles dominated by a few clear craters or a strong ridge the predictions line up with the ground truth, but on busier or low-contrast tiles the model under-detects, and the count error of ~ 6 objects per tile reflects this systematic shortfall.

6. Challenges and lessons learned

Most of the effort in this part of the project went not into the network but into the obstacles around it. I list them roughly in order of how much they hurt the final score.

Noisy semantic-to-instance conversion. The labels are semantic coverage masks, not object annotations. Connected-component labelling turns them into instances, but it is a leaky abstraction: two touching craters merge into one component, a single elongated ridge fragments into several, and “coverage-like” channels produce blobs that are not really countable objects at all. The area/shape filters and the 20% coverage cut-off I added help, but the supervision the detector sees is fundamentally noisier than in a hand-annotated detection dataset, and that noise puts a ceiling on achievable precision.

Severe class imbalance and rarity. The classes that matter most scientifically are the rarest in the data. Lobate scarps were too infrequent for the model to learn at all ($AP = 0$), and pit/skylights — the whole motivation for the task — appear so seldom that the network almost never fires on them ($AP \approx 0.0003$). Frequency-based channel selection keeps training stable, but it cannot manufacture positive examples that are not there. This is the single biggest driver of the low mAP.

Small, low-contrast, scale-ambiguous features. Lunar features are small relative to the tile and often only marginally brighter or darker than their surroundings, which is very different from the COCO objects the backbone was pretrained on. Retuning the anchors to lunar scales (8–128 px) and keeping the native 256 px resolution both helped, but the domain gap between natural RGB photographs and grayscale lunar terrain remains a real handicap for a transfer-learned backbone.

Detection $\neq$ segmentation. The most instructive lesson is that good masks did not buy good detections. Mask IoU/Dice are high precisely because they are only measured on objects that

were already matched; the bottleneck is upstream, in the proposal and box stage actually finding and correctly classifying the regions. Improving the mask head further would not move the headline metric — the next gains have to come from recall.

Overfitting on limited positives. With relatively few labelled positives, the model began overfitting after only a handful of epochs (Fig. 1). Dropout, weight decay and augmentation slowed it but did not prevent it; more aggressive augmentation, class-balanced sampling, or simply more labelled data would likely help more than a bigger network.

Compute budget. Training on CPU made every experiment expensive — roughly half a day per full run — which is why the loop was engineered to be resumable and why hyper-parameter exploration was necessarily limited. Most of the design choices above were made to get the most out of a single, long, interruptible run rather than many short ones.

What I would change next. Given more time I would (i) address the imbalance directly with class-balanced or hard-negative sampling and a focal-style loss; (ii) treat the rare pit/skylight class as a dedicated detection problem rather than one head among four; (iii) move to GPU so the schedule can be tuned and stopped at the validation-loss minimum; and (iv) revisit the instance-generation step, since cleaner targets would raise the ceiling on everything downstream.

7. Conclusions

I adapted Mask R-CNN to instance-segment four classes of lunar features from Marius Hills tiles, contributing a channel-audited semantic-to-instance pipeline, a deeper custom box and residual mask head, and lunar-scale anchors, all trained in a crash-safe CPU loop. The model learns to outline features well (mask IoU 0.64, Dice 0.77) but detects them poorly (mAP@0.5 $\approx 1\%$), with the rare and most interesting classes essentially undetected. The clear separation between mask quality and detection quality, and the data-side reasons behind it, are the main takeaways and the right starting point for the next iteration.

Acknowledgements. This work was carried out for the Laboratory of Computational Physics (LCP) module B course (Physics of Data) at the University of Padua, using the course-provided Marius Hills dataset derived from LROC imagery.

References

- He, K., Gkioxari, G., Dollár, P., & Girshick, R. 2017, in Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2961–2969
- He, K., Zhang, X., Ren, S., & Sun, J. 2016, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778
- Laboratory of Computational Physics (LCP) module B, Physics of Data, UniPD. 2026, Marius Hills lunar feature dataset: pre-processed NPZ tiles, tile meta-data index (`index.csv`) and the supplied lunar image tile, Course-provided dataset, derived from LROC imagery
- Lin, T.-Y., Dollár, P., Girshick, R., et al. 2017, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2117–2125
- Loshchilov, I. & Hutter, F. 2019, in International Conference on Learning Representations (ICLR)
- NASA. 2023, Artemis and the Lunar South Pole: ShadowCam Imaging of Permanently Shadowed Regions, NASA feature articles and ShadowCam mission pages, <https://www.nasa.gov/>
- NASA/GSFC & Arizona State University. 2011, PIA13523: Lunar South Polar Region, NASA Photojournal, <https://photojournal.jpl.nasa.gov/catalog/PIA13523>

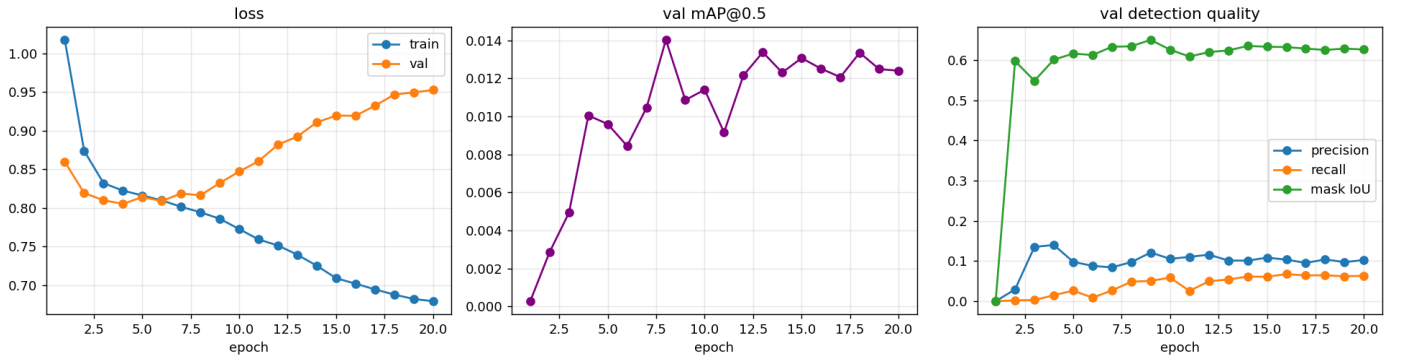


Fig. 1. Training history over 20 epochs. *Left:* train vs. validation loss — validation loss turns upward after ~ 4 epochs, indicating overfitting. *Centre:* validation mAP@0.5, peaking near epoch 8. *Right:* detection quality — mask IoU saturates early and high, while precision and recall stay low. The decoupling of mask quality from detection quality is the central result of this experiment.

NASA/JPL-Caltech & University of Arizona. 2010, PIA12954: A Hole in the Moon (Marius Hills Pit, a Possible Lava-Tube Skylight), NASA Photojournal, lunar Reconnaissance Orbiter Camera (LROC) NAC image, <https://photojournal.jpl.nasa.gov/catalog/PIA12954>

USGS Astrogeology Science Center & Arizona State University. 2024, Lunar Reconnaissance Orbiter Camera (LROC) Instrument Overview: Wide-Angle Camera (WAC) and Narrow-Angle Camera (NAC), USGS/LROC instrument documentation, <https://www.usgs.gov/>; <https://lroc.asu.edu/>

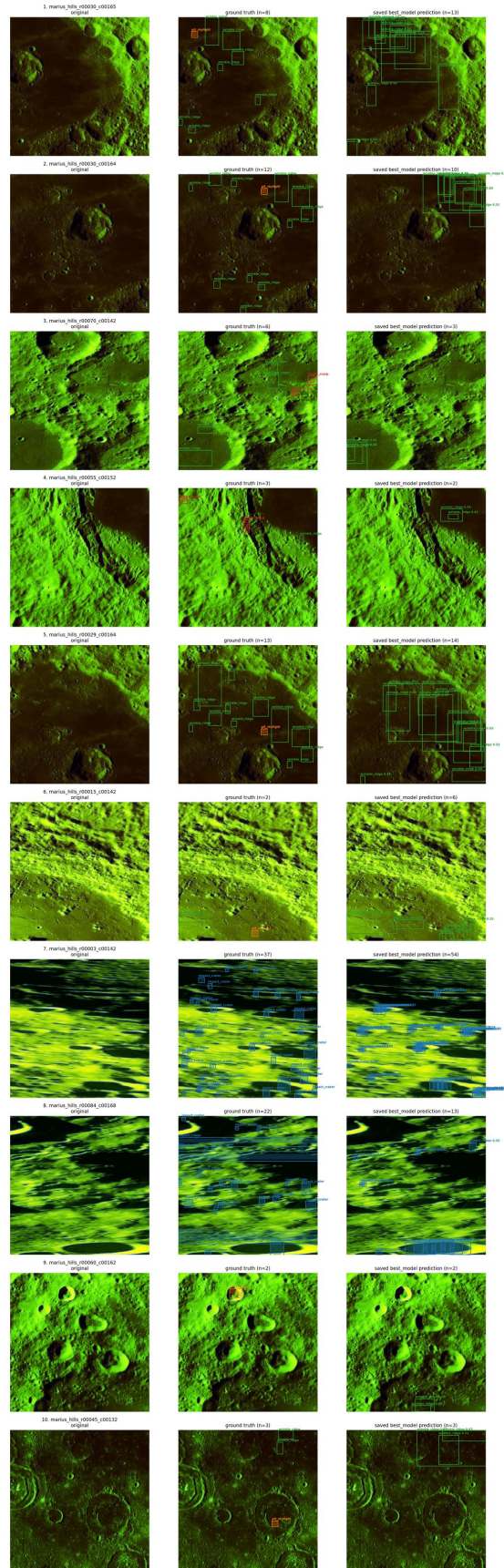


Fig. 2. Qualitative results on ten validation tiles from the Marius Hills region. For each tile: original image (left), ground-truth instances (centre), and model predictions with class and confidence (right). The network localises and outlines the larger, higher-contrast craters and ridges reasonably well, but misses faint or small features and is unreliable on the rare pit/skylight and lobate-scarp classes.